

# Expressions in Java

An expression in Java is a construct that evaluates to a single value. Expressions can be as simple as a single constant or variable, or as complex as a combination of variables, operators, method calls, and other expressions.

## Types of Expressions

### 1. Arithmetic Expressions:

- Combine variables and constants using arithmetic operators.
- Example:

```
int sum = 10 + 5;  
int product = sum * 2;
```

### 2. Relational Expressions:

- Compare two values and return a boolean result (**true** or **false**).
- Example:

```
boolean isEqual = (10 == 10); // true  
boolean isGreater = (10 > 5); // true
```

### 3. Logical Expressions:

- Combine boolean values using logical operators.
- Example:

```
boolean result = (10 > 5) && (5 < 10); // true
```

### 4. Assignment Expressions:

- Assign a value to a variable.
- Example:

```
int x = 5;  
x = x + 2; // x is now 7
```

### 5. Method Call Expressions:

- Call a method and optionally use its return value.
- Example:

```
int length = "Hello".length(); // length is 5
```

## Combining Expressions

Expressions can be combined to form more complex expressions:

```
int a = 10;  
int b = 20;  
int c = (a + b) * 2; // c is 60  
boolean isPositive = (c > 0) && (a < b); // isPositive is true
```

## Statements in Java

A statement in Java is a complete unit of execution. Statements are executed in sequence and can control the flow of the program.

### Types of Statements

#### 1. Declaration Statements:

- Declare a variable and optionally initialize it.
- Example:

```
int number;  
String message = "Hello, World!";
```

#### 2. Expression Statements:

- Perform an action such as assignment or method call.
- Example:

```
number = 10;  
System.out.println(message);
```

#### 3. Control Flow Statements:

- Control the order in which statements are executed.

##### a. Conditional Statements

- Execute different blocks of code based on conditions.
- Example:

```
if (number > 0) {
    System.out.println("Positive number");
} else {
    System.out.println("Non-positive number");
}

switch (number) {
    case 1:
        System.out.println("One");
        break;
    case 2:
        System.out.println("Two");
        break;
    default:
        System.out.println("Other number");
}
```

## b. Looping Statements

- Repeat a block of code multiple times.
- Example:

```
for (int i = 0; i < 5; i++) {
    System.out.println("i is " + i);
}

int i = 0;
while (i < 5) {
    System.out.println("i is " + i);
    i++;
}

i = 0;
do {
    System.out.println("i is " + i);
    i++;
} while (i < 5);
```

## c. Jump Statements

- Alter the flow of execution by jumping to another part of the program.
- Example:

```
for (int i = 0; i < 10; i++) {
    if (i == 5) {
        break; // exits the loop
    }
}
```

```
    if (i % 2 == 0) {  
        continue; // skips the current iteration  
    }  
    System.out.println("i is " + i);  
}  
  
public int add(int a, int b) {  
    return a + b; // exits the method  
}
```

## Code Blocks in Java

A code block is a group of zero or more statements enclosed in braces `{}`. Code blocks are used to group statements together and define the scope of variables.

### Types of Code Blocks

#### 1. Method Block:

- Defines the body of a method.
- Example:

```
public void printMessage() {  
    System.out.println("Hello, World!");  
}
```

#### 2. Loop Block:

- Defines the body of a loop.
- Example:

```
for (int i = 0; i < 5; i++) {  
    System.out.println("i is " + i);  
}
```

#### 3. Conditional Block:

- Defines the body of a conditional statement.
- Example:

```
if (a > 5) {  
    System.out.println("a is greater than 5");  
} else {  
    System.out.println("a is less than or equal to 5");  
}
```

#### 4. Anonymous Block:

- Groups statements without being associated with any control flow or method.
- Example:

```
{  
    int a = 10;  
    System.out.println("a is " + a);  
}
```

#### Nesting Code Blocks

Code blocks can be nested within each other, and this nesting defines the scope of variables.

##### Example:

```
public void exampleMethod() {  
    int outerVariable = 10;  
  
    if (outerVariable > 5) {  
        int innerVariable = 20;  
        System.out.println("Inner variable: " + innerVariable);  
    }  
  
    // The following line would cause an error because innerVariable is  
    // not visible here  
    // System.out.println("Inner variable: " + innerVariable);  
}
```

#### Conclusion

Expressions, statements, and code blocks are fundamental building blocks of Java programs.

Understanding how to use them effectively allows you to write clear, concise, and correct Java code.

- **Expressions:** Evaluate to a single value and can be simple or complex.
- **Statements:** Complete units of execution that control the flow of a program.
- **Code Blocks:** Group statements and define variable scope.